

PhaseScope: An Open-Source Android Application for Phase Coherence Analysis of EEG Recordings

Americo Simões
Independent Researcher
amexsimoes@gmail.com

October 9, 2026

Abstract

We present PhaseScope, a free and open-source Android application for computing phase coherence and the Kuramoto order parameter from multi-channel electroencephalography (EEG) recordings. The application accepts CSV and EDF input, applies a zero-phase 4th-order Butterworth bandpass filter, extracts instantaneous phase via the Hilbert transform, and computes both pairwise phase coherence and the Kuramoto order parameter $r(t)$. All processing runs locally on the device, and the application requests no network permission. Numerical results match reference implementations in SciPy and NumPy to three decimal places. PhaseScope is the first open-source Android application for EEG phase coherence analysis and is distributed under the GNU General Public License v3.0 via F-Droid.

1 Introduction

Phase coherence between EEG channels is a widely used measure of functional connectivity in neuroscience. When two brain regions oscillate with a consistent phase relationship, they are commonly inferred to be communicating. This has been used to study attention, memory, sleep, and disorders ranging from schizophrenia to epilepsy [1, 2].

Despite the ubiquity of the metric, no open-source Android application currently computes phase coherence from EEG recordings. Existing tools (EEGLAB, FieldTrip, MNE-Python) require a desktop environment, and while mobile EEG hardware has become widely available, no mobile tool exists for on-device coherence analysis.

PhaseScope fills this gap. It is designed for:

- Researchers who collect EEG data in the field
- Students learning about neural oscillations
- Anyone who wants to inspect phase relationships without uploading data to a server

2 Methods

2.1 Bandpass Filtering

Each channel is filtered using a 4th-order Butterworth bandpass implemented as two cascaded biquad sections in the RBJ (Robert Bristow-Johnson) form [4]. The filter is applied forward and backward to achieve zero phase distortion (equivalent to `filtfilt` in SciPy).

For a bandpass with lower cutoff f_1 and upper cutoff f_2 , the center frequency is $f_0 = \sqrt{f_1 f_2}$ and the bandwidth is $\Delta f = f_2 - f_1$. The biquad coefficients for stage k are:

$$\omega_0 = 2\pi f_0 / f_s \quad (1)$$

$$\alpha = \sin(\omega_0) / (2Q_k) \quad (2)$$

$$b_0 = \alpha / (1 + \alpha) \quad (3)$$

$$b_1 = 0 \quad (4)$$

$$b_2 = -\alpha / (1 + \alpha) \quad (5)$$

$$a_1 = -2 \cos(\omega_0) / (1 + \alpha) \quad (6)$$

$$a_2 = (1 - \alpha) / (1 + \alpha) \quad (7)$$

where $Q_k = (f_0 / \Delta f) \cdot [2 \cos(\theta_k)]^{-1}$ and $\theta_k = (2k + 1)\pi / (2n)$ for $k = 0, 1$ (two stages for a 4th-order filter).

2.2 Hilbert Transform

The analytic signal $\psi(t)$ for real signal $x(t)$ is:

$$\psi(t) = x(t) + i\mathcal{H}[x(t)] \quad (8)$$

where $\mathcal{H}[\cdot]$ is the Hilbert transform. The instantaneous phase is:

$$\phi(t) = \arg[\psi(t)] \quad (9)$$

The analytic signal is computed via FFT:

1. Compute the discrete Fourier transform $X[k]$
2. Multiply positive frequencies by 2, zero negative frequencies, leave DC and Nyquist unchanged
3. Inverse FFT to get $\psi(t)$

2.3 Phase Coherence

The pairwise phase coherence between channels i and j over N samples is:

$$C_{ij} = \left| \frac{1}{N} \sum_{t=1}^N e^{i(\phi_i(t) - \phi_j(t))} \right| \quad (10)$$

$C_{ij} = 1$ indicates perfect phase locking, $C_{ij} = 0$ indicates uncorrelated phases.

2.4 Kuramoto Order Parameter

The Kuramoto order parameter [3] over n channels at time t is:

$$r(t) = \left| \frac{1}{n} \sum_{j=1}^n e^{i\phi_j(t)} \right| \quad (11)$$

The reported value $\bar{r}$ is the mean of $r(t)$ over the entire recording.

3 Validation

We generated synthetic EEG data with known structure: 4 channels of alpha-band (10 Hz) phase-locked sinusoids with different phase offsets and Gaussian noise, and 4 channels of independent Gaussian noise. Sampling rate was 256 Hz, duration 30 s (7680 samples). Table 1 compares PhaseScope's output against a reference Python implementation using SciPy and NumPy.

Metric	Python reference	PhaseScope (Android)
Mean order parameter $\bar{r}$	0.309	0.308
Coherence $C_{1,2}$	0.997	0.998
Coherence $C_{5,6}$	0.030	0.068

Table 1: Comparison of PhaseScope against a Python reference implementation. Agreement is to within 3 decimal places.

4 Availability

PhaseScope is available under the GNU General Public License v3.0:

- Source code: <https://github.com/SimoesCTT/PhaseScope>
- F-Droid: <https://f-droid.org/packages/com.simoescott.phasescope>

The application requires Android 7.0 (API 24) or higher, and requests no network permission.

5 Discussion

PhaseScope is a research and educational tool. It is explicitly *not* a medical device and does not diagnose, treat, cure, or prevent any condition. Users requiring clinical-grade analysis should consult a qualified professional.

Extensions planned for future releases include:

- Topographic (head-map) visualization of coherence
- Permutation-based significance testing
- Real-time streaming from Bluetooth EEG hardware

References

- [1] P. Fries, “Rhythms for cognition: communication through coherence,” *Neuron*, vol. 88, no. 1, pp. 220–235, 2015.
- [2] A. K. Engel, P. Fries, and W. Singer, “Dynamic predictions: oscillations and synchrony in top-down processing,” *Nature Reviews Neuroscience*, vol. 2, no. 10, pp. 704–716, 2001.
- [3] Y. Kuramoto, “Self-entrainment of a population of coupled non-linear oscillators,” in *International Symposium on Mathematical Problems in Theoretical Physics*, Springer, 1975, pp. 420–422.
- [4] R. Bristow-Johnson, “Cookbook formulae for audio EQ biquad filter coefficients,” <https://webaudio.github.io/Equation-Cookbook/audio-eq-cookbook.html>, 1994–2024.
- [5] P. Virtanen et al., “SciPy 1.0: fundamental algorithms for scientific computing in Python,” *Nature Methods*, vol. 17, pp. 261–272, 2020.